VÕ TIẾN
Thảo luận kiến thức CNTT trường BK về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

Cấu Trúc Dữ Liệu và Giải Thuật (DSA)

DSA - HK243

GK - Tree Basic

Thảo luận kiến thức CNTT trường BK
về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

TP. HỒ CHÍ MINH, THÁNG 7/2025

Mục lục

1 Lý thuyết cơ bản Tree **2**
1.1 Tree là gì? 2
1.2 Tree Traversals (Duyệt cây) 2

2 Câu hỏi trong các đề thi **3**

1 Lý thuyết cơ bản Tree

1.1 Tree là gì?

Tree (Cây) là cấu trúc dữ liệu dạng phân cấp, mỗi phần tử gọi là **nút (node)**. Một cây có:

- **Nút gốc (Root)**: nút không có cha.
- **Nút lá (Leaf)**: nút không có con.
- **Nút cha - Nút con**: liên kết bởi cạnh.

1.2 Tree Traversals (Duyệt cây)

Depth-First Search (DFS):

- **Preorder**: Nút cha → Con trái → Con phải.
- **Inorder**: Con trái → Nút cha → Con phải.
- **Postorder**: Con trái → Con phải → Nút cha.

```cpp
// Cấu trúc cơ bản của Node trong cây nhị phân
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

// Duyệt tiền thứ tự (Preorder)
void preorder(Node* root) {
    if (root == nullptr) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

// Duyệt trung thứ tự (Inorder)
void inorder(Node* root) {
    if (root == nullptr) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

// Duyệt hậu thứ tự (Postorder)
void postorder(Node* root) {
    if (root == nullptr) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

2 Câu hỏi trong các đề thi

1. Số nút tối đa của một cây nhị phân có chiều cao h?

   a) h b) $2^{h-1}+1$ c) 2^h-1 d) $2^{h+1}+1$

   Đáp án: **C** — Công thức tổng quát: 2^h-1

2. Số node tối thiểu của một cây nhị phân có chiều cao h là:

   a) h b) $h+1$ c) 2^{h-1} d) $2^{h-1}+1$

   Đáp án: **A** — Cây dạng chuỗi có h node và chiều cao h

3. Cho một cây nhị phân có chiều cao $H=5$. Gọi N là số nút trên cây thì $A \le N \le B$. Giá trị của $B+A$ là:

   a) 36 b) 55 c) 63 d) 94

   Đáp án: **A** — Min $A=5$ (cây chuỗi), Max $B=31$ (cây đầy đủ: 2^5-1), $A+B=36$

4. Cho các phát biểu sau đây:

   (a) Tất cả các nút trong một cây đều có bậc nội (indegree) bằng 1. **(Sai – trừ gốc)**

   (b) Trong một cây nhị phân, một nút bất kỳ có thể có ít hơn 2 con. **(Đúng)**

   (c) Tất cả các nút (ngoại trừ nút lá) trong một cây đều có bậc ngoại lớn hơn hoặc bằng 1. **(Đúng)**

   (d) Trong một cây nhị phân, nếu biết chiều cao của cây là 4 thì số nút của cây là 15. **(Sai – chỉ đúng với cây đầy đủ)**

   a) 0 b) 1 c) 2 d) 3

   Đáp án: **C**

5. Cây nhị phân có 100 node, tính chiều cao nhỏ nhất:

   a) 7 b) 5 c) 6 d) 8

   Đáp án: **C** — $2^6=64$, $2^7=128$ → cần ít nhất chiều cao $h=7$ để chứa 100 node → $h_{min}=7$

6. Cây nhị phân có 100 node, chiều cao lớn nhất:

   a) 100 b) 99 c) 101 d) 98

   Đáp án: **A** — Cây dạng chuỗi có chiều cao $=n=1000$

7. Cây nhị phân có 100 node, số node lá của cây:

   a) `[1:100]` b) `[2:99]` c) `[1:50]` d) `[1:55]`

   Đáp án: **C** — Số lá tối thiểu là 1, tối đa là $\left\lfloor \frac{n+1}{2} \right\rfloor = 50$

8. Cây nhị phân có 100 node, số lượng cây có thể là cây có chiều cao lớn nhất:

   a) 2^{100} b) 2^{99} c) 1 d) 100

   Đáp án: **B** — Mỗi node (trừ node cuối) có thể chọn trái/phải → 2^{99} khả năng

9. `[HK241&HK242]` Trong cây nhị phân, độ sâu (depth) của một node là:

   a) Số lượng nút con trực tiếp của nút đó
   b) Số lượng nút cha trực tiếp của nút đó
   c) Số lượng nút con trực tiếp và gián tiếp của nút đó
   d) Số lượng nút cha trực tiếp và gián tiếp của nút đó

   Đáp án: **D** — Độ sâu = số lượng tổ tiên (cha, ông, cụ...)

10. Cho các phát biểu sau về cây nhị phân:

(a) Cây nhị phân đầy đủ là cây mà tất cả các nút đều có 0 hoặc 2 con. **(Đúng)**

(b) Cây nhị phân hoàn chỉnh là cây mà tất cả các mức (trừ mức cuối cùng) đều đầy đủ, và các nút ở mức cuối được lấp từ trái sang phải. **(Đúng)**

(c) Mọi cây nhị phân hoàn chỉnh đều là cây nhị phân đầy đủ. **(Sai — cây hoàn chỉnh có thể có 1 con ở mức cuối)**

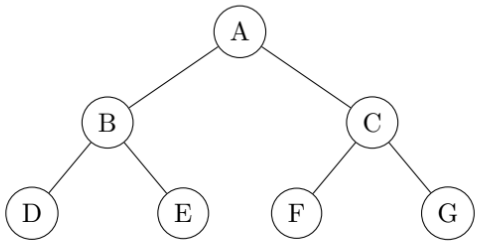
(d) Cây nhị phân gần hoàn chỉnh là một cách gọi khác của cây nhị phân hoàn chỉnh. **(Sai — không phải thuật ngữ chuẩn hoặc chính thức)**

Số phát biểu đúng là:

a) 1 b) 2 c) 3 d) 4

Đáp án: B

11. Cho cây nhị phân sau:



Chọn các phát biểu đúng:

(a) Duyệt trung thứ tự (in-order): D, B, E, A, F, C, G

(b) Duyệt tiền thứ tự (pre-order): A, B, D, E, C, F, G

(c) Chiều cao của cây là 2

(d) Cây có 4 nút lá và 3 nút nội (tính luôn node root)

a) 1 b) 2 c) 3 d) 4

Đáp án: C

- A — đúng: Duyệt trung thứ tự là: trái, gốc, phải → D, B, E, A, F, C, G.
- B — đúng: Duyệt tiền thứ tự là: gốc, trái, phải → A, B, D, E, C, F, G.
- C — sai: Chiều cao = 3, n
- D — đúng: 4 lá: D, E, F, G. 3 nút nội: A, B, C.

12. [HK241&HK242] Chọn hiện thực đúng của hàm `prtIn` xuất cây nhị phân theo duyệt cây trung thứ tự (in-order traversal):

a) `if (root != NULL){ cout « root->data; prtIn(root->left); prtIn(root->right);}`
b) `if (root != NULL){ cout « root->data; prtIn(root->right); prtIn(root->left);}`
c) `if (root != NULL){ prtIn(root->left); cout « root->data; prtIn(root->right);}`
d) `if (root != NULL){ prtIn(root->right); prtIn(root->left); cout « root->data;}`

Đáp án: C — Thứ tự trung: `left → node → right`.

13. Chọn hiện thực đúng của hàm `prtPost` xuất cây theo duyệt hậu thứ tự (post-order traversal):

a) `if (root != NULL){ prtPost(root->left); prtPost(root->right); cout « root->data;}`
b) `if (root != NULL){ cout « root->data; prtPost(root->left); prtPost(root->right);}`
c) `if (root != NULL){ prtPost(root->right); cout « root->data; prtPost(root->left);}`

d) `if (root != NULL){ prtPost(root->left); cout « root->data; prtPost(root->right);}`

Đáp án: **A** — Thứ tự hậu: `left → right → node`.

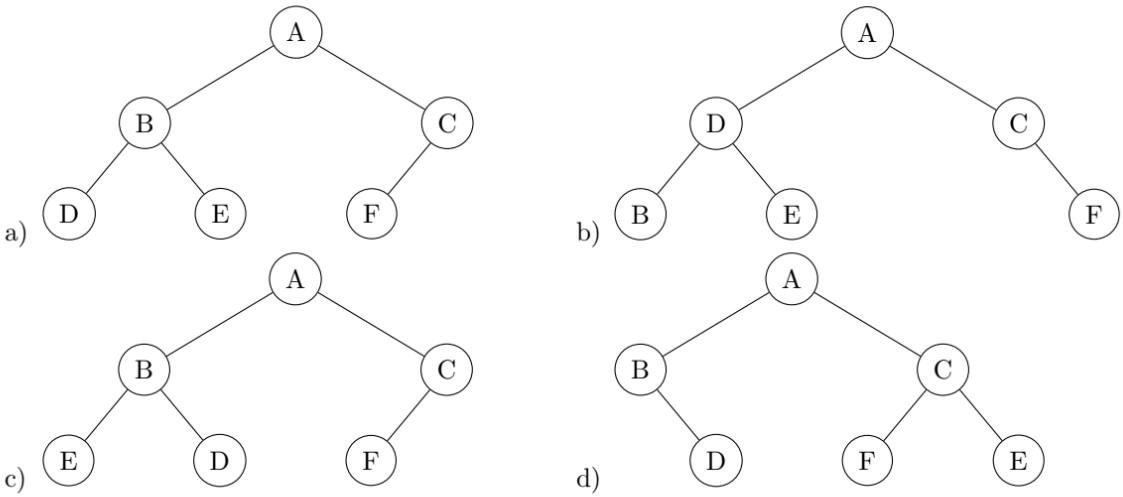
14. Chọn hiện thực đúng của hàm `prtPre` xuất cây theo duyệt tiền thứ tự (pre-order traversal):

    a) `if (root != NULL){ cout « root->data; prtPre(root->left); prtPre(root->right);}`
    b) `if (root != NULL){ prtPre(root->left); cout « root->data; prtPre(root->right);}`
    c) `if (root != NULL){ prtPre(root->right); prtPre(root->left); cout « root->data;}`
    d) `if (root != NULL){ prtPre(root->left); prtPre(root->right); cout « root->data;}`

    Đáp án: **A** — Thứ tự tiền: `node → left → right`.

15. `[HK241&HK242]` Cây nhị phân nào sau đây phù hợp với các kết quả duyệt sau?

    - Trung thứ tự (in-order): D, B, E, A, F, C
    - Hậu thứ tự (post-order): D, E, B, F, C, A



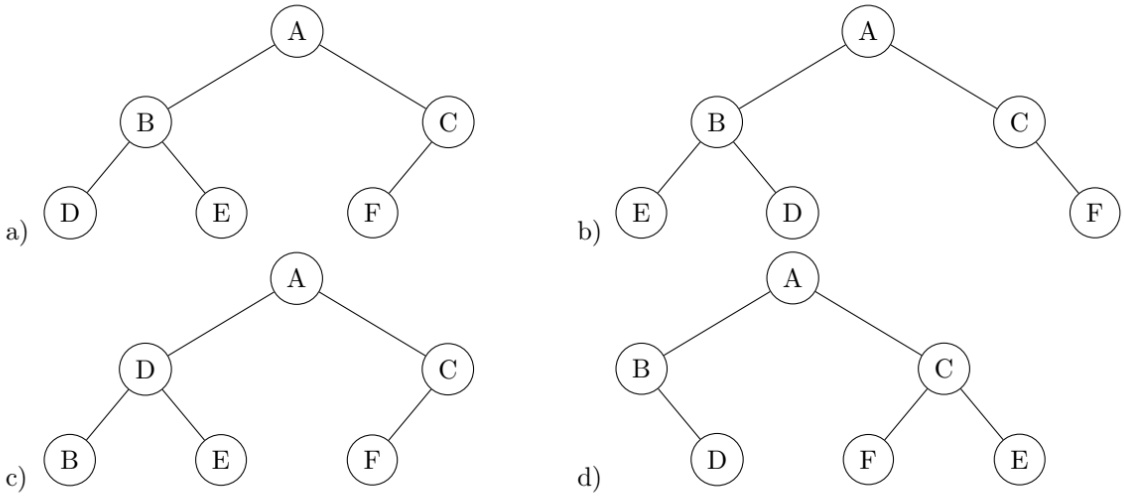
Đáp án: **A**

- Post-order kết thúc với A -> A là root
- Trong in-order, dãy D, B, E nằm trước A đó là cây con trái
- D, E là con của B (theo post-order: D, E, B)
- F, C là con phải của A (F là con trái của C theo in-order/post-order)

Cây A là cấu trúc đúng theo hai dãy duyệt.

16. Cây nhị phân nào sau đây phù hợp với các kết quả duyệt sau?

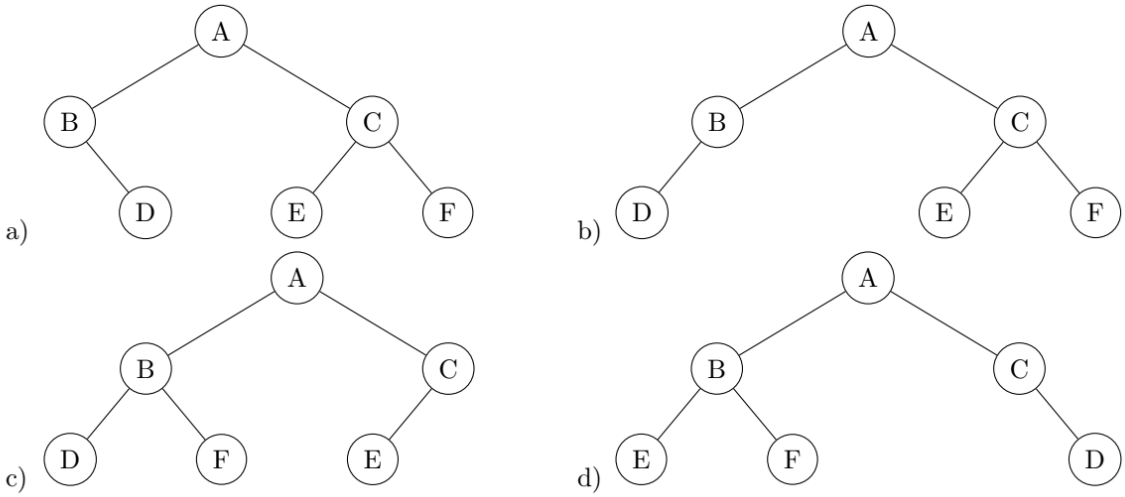
    - Tiền thứ tự (pre-order): A, B, D, E, C, F
    - Trung thứ tự (in-order): D, B, E, A, F, C



Đáp án: A

- Pre-order bắt đầu với A → A là root
- B là con trái của A; từ in-order: D, B, E là cây con trái
- F, C là phần còn lại (F là con trái của C theo in-order)

17. Cây nhị phân nào sau đây phù hợp với các kết quả duyệt sau?
    - Tiền thứ tự (pre-order): A, B, D, C, E, F
    - Hậu thứ tự (post-order): D, B, E, F, C, A



Đáp án: B

- Pre-order: root là A → con trái là B (với D), con phải là C (với E, F)
- Post-order: D, B, E, F, C, A → đúng cấu trúc theo thứ tự con trước, gốc sau

18. Cho hàm sau:

```
int countL0(Node *root) {
    if (!root == NULL) return 0;
    // Code
    return countL0(root->left) + countL0(root->right) + 1;
}
```

Điền vào `// Code` để hoàn thành hàm trả về số node lá trong cây nhị phân.

a) `if (!root->left && !root->right) return 1;`

b) `if (!root->left || !root->right) return 1;`
c) `if (!root->left && !root->right) return 0;`
d) `if (!root->left || !root->right) return 0;`

Đáp án: **A** — Vì nếu nút đó không có cả trái và phải thì đó là lá → trả về 1.

19. [HK241&HK242] Cho lớp giao diện sau mô tả cấu trúc một nút trong cây nhị phân:

```
template<typename E>
class BNode {
    virtual E &element() = 0; // trả về giá trị tại nút
    virtual BNode<E> *left() const = 0; // trả về con trái
    virtual BNode<E> *right() const = 0; // trả về con phải
};
```

Hàm nào sau đây trả về chiều cao của cây nhị phân?

(a)
```
int height(BNode<int>* root) {
    if (root == nullptr) return 0;
    return 1 + max(height(root->left()), height(root->right()));
}
```

(b)
```
int height(BNode<int>* root) {
    if (root == nullptr) return 0;
    return 1 + min(height(root->left()), height(root->right()));
}
```

(c)
```
int height(BNode<int>* root) {
    if (root == nullptr) return 1;
    return max(height(root->left()), height(root->right()));
}
```

(d)
```
int height(BNode<int>* root) {
    return max(height(root->left()), height(root->right())) + 1;
}
```

Đáp án: **A** - Với mỗi nút, chiều cao = 1 + max(chiều cao con trái, chiều cao con phải).